

Beyond Prices and News: A Novel Deep Learning Framework for Stock Forecasting

Leonardo De Marchi[†]

Abstract—It’s been years that financial forecasting modeling and the so-called quants have stole the show in investments decisions and risk management applications, triggering hedge funds, big assets managers and fintech companies worldwide to enter the race. The modern techniques and frameworks to gather, process and analyze big amounts of data are unprecedented in human history, making this relatively new field so attractive and rapidly growing. This work presents a better, more robust approach to [1], exploiting its novel idea to include textual information alongside financial data, but taking it further. The model combines, via a blending ensemble learning method, the predictions of two recurrent neural networks followed by a fully connected neural network. Using the S&P500 Index as test case, this paper aims to demonstrate that the market state can be described - and hence predicted - with improved accuracy compared to prior approaches. The current belief of the author is that every movement in large markets is interconnected, therefore having access to comprehensive political, fiscal, and economic indicators is expected to lead to more accurate predictions. The proposed Beyond Prices and News framework outperforms standard deep learning baselines, reducing MSE by up to 33% and improving F1-Score by more than 40% compared to LSTM and GRU architectures. Furthermore, when compared to a two-feature version of the same model, the full-feature design achieves a 7% reduction in MSE and 26% improvement in directional accuracy (MDA), underlining the benefits of richer data integration.

Index Terms—Financial Forecasting, Stock Markets, Ensemble Learning, Recurrent Neural Networks, Natural Language Processing.

I. INTRODUCTION

Nowadays, financial and stock markets play a prominent role in the global economy. Accurate forecasting of their dynamics is a long-standing and challenging problem, eased in recent years by advances in artificial intelligence and the availability of large scale data, with deep learning methods becoming increasingly popular.

Algorithmic trading, summarizeable as the use of computer algorithms to automatically manage orders in financial markets, has become a dominant force in today’s exchanges. Within this landscape, high-frequency trading (HFT) represents a subset characterized by ultra-fast execution and massive order flows. According to a working paper by the U.S. Securities and Exchange Commission (SEC), in 2015 such activity already accounted for roughly 55% of the total trading volume in U.S. equity markets [2]. This implies that a substantial share of

market movements is now driven by algorithmic predictions, whether accurate or not. Ensuring that predictive models capture meaningful signals rather than noise is therefore crucial, as poor forecasts can easily lead to erratic market behavior.

The proposed framework addresses these challenges by building on the weak points of [1], which relied exclusively on prices and sentiment score extracted from financial news. While textual information may indeed contain political, fiscal, and other relevant signals, relying solely on these two inputs risks missing out on important predictive dimensions. Hence, a richer set of features has been integrated, carefully chosen to avoid overfitting. On the one hand, technical indicators derived from price dynamics provide valuable information about short and medium-term market trends. On the other hand, exogenous macro-financial indicators are included to capture broader market conditions. Furthermore, the framework is designed to be scalable: additional, less easily retrievable but highly informative variables can be incorporated to further enhance predictive power.

Major contributions with respect to [1]:

- **Richer dataset:** A dataset with a larger number of samples and a more comprehensive set of features is constructed.
- **Improved preprocessing:** Log returns are used instead of raw prices, ensuring stationarity and reducing scale effects in the input data.
- **Model tuning:** Systematic hyperparameter tuning through cross-validation is applied, to enhance robustness and generalization.

This report is structured as follows. In Section II the state of the art gets described, the system and data models are respectively presented in Sections III and IV. The proposed signal processing technique is detailed in Section V and its performance evaluation is carried out in Section VI, as well as some concluding remarks.

II. RELATED WORK

Classical statistical models, such as ARIMA [3] and GARCH [4], as well as traditional machine learning methods (e.g. SVMs [5], random forests [6]) have long been applied to financial forecasting. These approaches provide solid baselines, but often lack of, respectively, adaptability and scalability.

With the rise of deep learning, recurrent architectures such as Long Short-Term Memory (LSTM) [7] and Gated Recurrent Units (GRU) [8] can learn more complex and time dependent

[†]Department of Information Engineering, University of Padova,
email: leonardo.demarchi.2@studenti.unipd.it,
ID number: 2158290

patterns in data. Convolutional networks have also been employed in time-series forecasting [9], though their adoption in finance has been more limited compared to recurrent models. A major milestone in deep learning was the introduction of the attention mechanism and the Transformer architecture [10], which led to an unprecedented complex-pattern capturing ability and the possibility to track very long and distant relationships (as it can be seen with the extraordinary advance of NLP models like LLMs). Therefore hybrid models, which comprehends the latest techniques, represents now the state of the art for financial forecasting. Even the biggest players on the planet, like BlackRock - which is the largest asset manager in the world, responsible for tens of trillions of dollars - have long utilized predictive models, and in the past decade entered the big data and AI race, integrating their platforms with textual analysis and NLP.

To discuss whether this ever increasing centralization of markets manipulative power is a good sight or not, is not the aim of this report, but in this context [1] represents a notable step toward multimodal modeling, by integrating news-based sentiment with prices.

Its main contribution is to demonstrate that textual information can add predictive signal beyond market quotes through a compact, reproducible pipeline. Nevertheless, its feature space remains narrow, mostly prices plus sentiment scores. While news, as said, can carry relevant information about market behaviour, and for sure add semantic value to the price taken alone, they still may be biased depending on the newspaper, or not expressive enough, or misleading on purpose, or even difficult to retrieve.

In [1], Li and Pan propose possible improvements to their work such as hyperparameter tuning and integration of different data sources as the 10-K annual report. Still, the possibility of incorporating technical and macro-financial indicators is not considered, as well as modeling log returns instead of raw prices series, typically known to be non stationary and characterized by strong trends or biases. These limitations leave open whether a more targeted feature design can translate into measurable gains.

III. PROCESSING PIPELINE

Data Source

S&P500 Index has been chosen as case study as it represents quite accurately the dynamics of whole U.S. equity market, and is therefore widely adopted in the literature as a benchmark for financial forecasting models. The data used in this work comes from different sources:

Historical prices were retrieved from Yahoo Finance via `yfinance`, while external financial indicators were obtained from FRED - Federal Reserve Economic Data. News data comes from FNSPID, a comprehensive financial dataset [11]. It contains 15.7 million financial news records for 4,775 S&P500 companies from 1999 to 2023, gathered from four stock market news websites. While news data used in this work differ from those in the reference paper [1], as the original sources are not publicly available, the initial

approach was to employ data covering the same time period. As it will be object of deeper analysis in Section V, this approach revealed to be inconsistent and based on a dataset that was too limited for generalization.

A second approach leveraged a dataset roughly 10 times larger. Without going too far into the past, and trying to avoid the highly unpredictable outcomes of COVID-19, while still maintaining computational feasibility, chosen dates range from January 2015 up to the end of December 2018¹.

Chosen features include the Volatility Index (VIX) of Chicago Board Options Exchange (CBOE), known as a popular measure of investor's expected market volatility - hence risk aversion; Treasury yield spread (T10Y2Y), defined as the difference between the 10-year and 2-year U.S. Treasury yields, a widely used proxy for recession expectations, and West Texas Intermediate (WTI) crude oil price, as a proxy for global energy market conditions - often strongly correlated with overall market trends.

Pipeline

At a high level, besides formatting and preparing numerical data, preprocessing ensures that news data comes exclusively from trading days, excluding weekends or holidays when the New York Stock Exchange (NYSE) is closed. Only news titles are considered, since they usually contain the main sentiment indicators, while article bodies can be source of additional noise. See Fig. 1 for a quick overview of the complete processing pipeline.

News are then processed with FinBERT, a pre-trained deep learning Transformer based NLP model. It is built by further training the BERT language model specifically in the finance domain [12]. For this reason, it is preferred over VADER, as used by Li and Pan [1], whose more general knowledge may cause financial terms such as "short" or "liability" to be misunderstood.

Once final data are obtained, enriched with a total of 12 features accurately outspread in Section IV, three time series datasets are built. Samples are generated using a rolling window of trading days, with the prediction target set to the trading day that immediately follows the window. A fixed window size of 10 is used as in [1], being it a reasonable length to catch some older trends but also to be sufficiently responsive to faster changes.

Models chosen for this task are LSTM and GRU networks. They were designed in the 1990s specifically to learn across long time periods, managing flow of information through gates and easing vanishing gradients problem by allowing an easier backpropagation through time. Level 1 predictions are passed to a meta-learner, implemented here as a level 2 fully connected feedforward network. Its role is to learn how to combine the outputs of the base models in a data-driven way, potentially outperforming fixed averaging schemes.

¹As it was author late and unfortunate discovery, December 2018 revealed to be the worst single month of the decade for stock markets, with sharp losses reaching -20%. Further analysis will be done in Section VI.

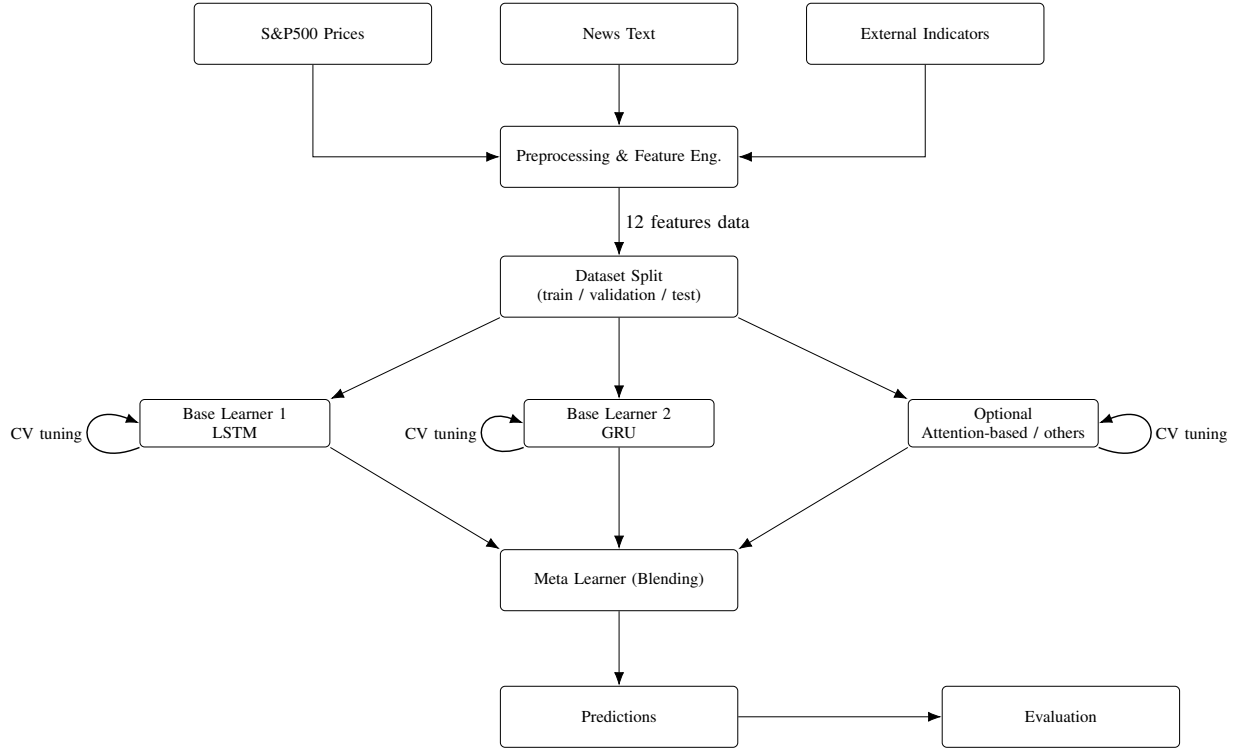


Fig. 1: High-level pipeline: from data sources to preprocessing, dataset split, base learners, blending meta-learner, and evaluation.

Hyperparameters are tuned using cross-validation with expanding training folds. This method is well suited for time series forecasting, since the training set grows over time while validation is carried out on later observations, better mimicking the temporal structure of the data.

The proposed framework addresses a regression task over the logarithmic returns (i.e., the logarithm of consecutive price ratios), which are more stationary, less biased and widely regarded as more semantically meaningful for financial modeling than raw prices. The aim of the optimization process is to minimize the Mean Squared Error (MSE) between the predicted and actual log return, forcing the models to capture patterns that reflect both the direction and the magnitude of price movements, rather than attempting to estimate the absolute price level itself. Directional accuracy is often of great help in defining trading strategies, where profitability in the vast majority of cases depends on anticipating correctly the sign of market changes. Therefore, while MSE is adopted as the primary loss metric, additional evaluation metrics are considered, including F1-score, precision, recall, and Mean Directional Accuracy (MDA).

Lastly, given the common regime shifts in markets volatility, to provide a comparable metric between different scenarios the Normalized Root Mean Squared Error (NRMSE) is also monitored. This is obtained dividing the Root Mean Squared Error by the standard deviation of test target data, thus providing a scale-independent error measure.

IV. SIGNALS AND FEATURES

The goal of this section is to clearly define how raw data are transformed into feature vectors, highlighting the contribution of each feature group to the forecasting task.

Both raw prices used (i.e., S&P500 and WTI) are transformed into logarithmic returns to improve stationarity and remove long-term upward trends, that models could otherwise exploit by trivially predicting an increase in most cases. The log-return at day t is defined as:

$$r_t = \log \left(\frac{P_t}{P_{t-1}} \right), \quad (1)$$

where P_t is the price at day t .

For prices, it's also useful to compute the standardized score (z-score) over a 20-day rolling window:

$$z_t = \frac{P_t - \mu_{t,20}}{\sigma_{t,20}}, \quad (2)$$

where $\mu_{t,20}$ and $\sigma_{t,20}$ are the mean and standard deviation of the last 20 trading days, respectively. The z-score provides a measure of how extreme a current price is relative to its recent history, which is useful to detect local overbought/oversold conditions.

For the S&P500 prices only, moving averages are further included, respectively of length 10 and 50 trading days (MA_{10} , MA_{50}). These are widely used in technical analysis to capture short and medium-term trends and to identify potential momentum shifts.

Two additional technical indicators are also considered for the

S&P500, computed respectively on prices and log returns:

- The Relative Strength Index (RSI) with a 14-day window, defined as

$$RSI_{14} = 100 - \frac{100}{1 + RS}, \quad (3)$$

where RS is the ratio of average gains to average losses over the past 14 days. The RSI is commonly used to assess momentum and to signal overbought or oversold conditions.

- The 20-day rolling volatility is computed as the standard deviation of log-returns over the last 20 trading days, annualized by multiplying by $\sqrt{252}$ (where 252 is the number of trading days in a year). This adds information about regime volatility clustering (calm vs. turbulent periods) that log returns alone do not capture well.

As already discussed in Section III, in addition to price-derived indicators, exogenous financial signals and sentiment score are integrated. For the formers, specifically

- The CBOE Volatility Index (VIX);
- The term spread ($T10Y2Y$);
- The daily change in the term spread ($\Delta T10Y2Y$), to capture short-term shifts in market expectations about future economic conditions.

In order to reduce asymmetries in the input signals, several additional transformations are applied.

First, instead of using raw moving averages, percentage differences from the adjusted closing price are computed:

$$MA_{k,pct}(t) = \frac{MA_k(t) - P_t}{P_t}, \quad k \in \{10, 50\}, \quad (4)$$

where $MA_k(t)$ is the k -day moving average at time t and P_t the adjusted closing price. Expressing moving averages as relative deviations rather than absolute price levels makes the series stationary and avoids scale issues.

Second, rolling volatility is log-transformed, to reduce the impact of extreme outliers. A similar log transformation is applied to the VIX index to smooth out its heavy-tailed distribution. Finally, since the learning task is carried out on log-returns, raw prices are dropped from the dataset. Table 1 shows an example of the resulting feature set for five consecutive trading days in January 2015.

To populate time series datasets, samples are generated using a rolling window approach.

The training set is used to train the level-1 learners, the validation set is used to obtain their predictions and to train the meta-learner on top of them, while the test set is reserved for out-of-sample evaluation of the entire framework. In this work, the training period spans from January 2015 to April 2018, totaling approximately 800 samples. The validation set covers April to August 2018, with 81 samples, and the test set spans from mid August to December 2018, with 82 samples. Within the process of creating datasets, data gets also normalized, i.e. rescaled from original range. This is done with different transformations depending on the feature type. For example, log returns are already centered around zero but are standardized to have unit variance. All other features

expressed on larger scales are likewise normalized to zero mean and unit variance. Exceptions are the z-scores, which are inherently standardized by construction, and the RSI. In the latter case, the indicator is rescaled with respect to its neutral level of 50, so that values above zero correspond to upward momentum and values below zero correspond to downward momentum. This transformation allows to preserve its conventional financial interpretation.

Crucially, scaling parameters (feature-wise mean and standard deviation) are computed exclusively on the training samples. This procedure prevents *data leakage*, i.e., the unintentional use of future information during training, that would lead to artificially improved in-sample performances.

V. LEARNING FRAMEWORK

The learning strategy adopted in this work consists of three main phases: hyperparameter tuning of the base learners, hyperparameter tuning of the meta-learner, and final training of the models with fixed configurations.

Tuning is performed with *Optuna*, an open source hyperparameter optimization framework to automate hyperparameter search.

For what it concerns the base models, a cross validation with expanding folds is performed, aiming at finding the best configuration in order to minimize the mean of the validation losses across folds. The search space includes the number of recurrent layers, hidden units per layer, dropout rate, learning rate, weight decay, and batch size. Hidden units and layers were kept within moderate ranges (e.g., 2-6 layers and tens of hidden units), since financial time series are notoriously noisy, and too deep models risk overfitting. Dropout was explored between 0.1 and 0.5 to regularize learning, and learning rates were varied over several orders of magnitude (10^{-4} to 10^{-2}), to account for the sensitivity of recurrent models to optimization dynamics. Weight decay was included to encourage generalization, and batch sizes were kept small, as large batches can hide local fluctuations that may carry predictive value in financial data. Fixed design choices regarded the optimizer as AdamW [13], and initialization schemes of the models. This variant of Adam decouples weight decay from the gradient-based adaptive updates, allowing for more effective regularization and often yielding better generalization, especially when data are scarce and noisy. For LSTM and GRU layers, input weights were initialized with Xavier uniform distribution, recurrent weights were initialized orthogonally (applied separately to each gate) to better explore parameters space, and biases were set to zero except for, respectively, the forget and update gate. Those biases were initialized to one to encourage the model to “forget less” at the start of training. In both architectures, linear layers were initialized with Xavier uniform weights and zero biases. These initialization strategies are regarded as good practice.

In a first attempt, tuning was carried out on a dataset limited to the same time horizon as in [1]. However, as introduced in Section III, results proved inconsistent. For the LSTM, the

TABLE 1: Examples of transformed features (January 2015).

Date	log_ret	rsi14	zscore_20	wti_log_ret	wti_zscore_20	t10y2y	dt10y2y	score	ma10_pct	ma50_pct	vol20_log	vix_log
2015-01-23	-0.0055	52.84	0.31	-0.0157	-1.20	1.29	-0.08	0.084	-0.0112	-0.0024	-1.89	2.81
2015-01-26	0.0026	54.05	0.56	-0.0097	-1.25	1.29	0.00	0.076	-0.0131	-0.0048	-1.89	2.74
2015-01-27	-0.0135	47.25	-0.37	0.0236	-0.83	1.29	0.00	0.074	0.0004	0.0086	-1.85	2.85
2015-01-28	-0.0136	41.63	-1.37	-0.0393	-1.37	1.23	-0.06	0.098	0.0130	0.0221	-1.81	3.02
2015-01-29	0.0095	46.41	-0.48	0.0018	-1.30	1.26	0.03	0.098	0.0039	0.0122	-1.78	2.93

mean validation loss across folds was 1.33, with individual fold values ranging from 0.58 to 2.17. For the GRU, the mean validation loss was 1.40, with folds ranging from 0.59 to 2.55. Moreover, the “best epochs” estimated by cross-validation were very low (median of 4 for LSTM and 7 for GRU), indicating that models were effectively under-trained.

To overcome these limitations the same tuning procedure was repeated on a dataset roughly ten times larger. This allowed validation folds to cover broader market conditions and provided more reliable metrics. For the LSTM, the mean validation loss decreased to 1.03, while for the GRU it was 1.06. Although one fold in each case displayed very high error (above 3.0), likely due to a sudden regime shift, the overall behavior was more representative. Hence, this scenario is the one to consider for this work, while tuning search space was slightly reduced for computational feasibility.

Once the level-1 predictions were obtained, the meta-learner was also tuned using Optuna. Here, a simpler 80/20 train-validation split was adopted, as the meta-learner operates on aggregated predictions rather than time series. The tuning focused on finding an adequate network depth, regularization level, learning rate and batch size design to combine base learner outputs effectively. Fixed choices were AdamW and LeakyReLU as activation function, usually able to improve dying gradients problem from standard ReLU and achieve satisfying results on rather small datasets avoiding the complexity of ELU.

After selecting the best hyperparameters, final training was run for a fixed number of epochs: 100 for the base learners, following the setup of [1], and 180 for the meta-learner.

The former is adopted to provide the models an adequate training process, as early stopping signals, in this context, have already been proved to be unreliable. The latter instead comes directly from the tuning process. Here, train and validation splits are made of base models predictions, retaining a more structured representation hence making early stopping signals more consistent.

VI. RESULTS AND CONCLUDING REMARKS

As presented in Section III, besides MSE and NRMSE, classification metrics such as Precision, Recall and F1-score get evaluated, as well as Mean Directional Accuracy (MDA). Therefore, both predicted and test logarithmic returns are converted into positive class if the value was positive, labeled as 1, and negative class otherwise, labeled as 0.

Precision intuitively accounts for the ability of the model not to label as positive a sample that is negative. Recall measures the ability to find all the positive samples, while the F1-score can be interpreted as the harmonic mean of precision and

recall. For all these metrics, the best value is 1 and the worst is 0.

MDA is computed as

$$\text{MDA} = \frac{1}{T} \sum_{t=1}^T \mathbf{1}[\text{sign}(\hat{y}_t) = \text{sign}(y_t)],$$

that is, the average fraction of times the predicted return $\hat{y}_t$ has the same sign as the true return y_t .

The Mean Prediction Accuracy (MPA) is usually computed on prices, measuring the average relative deviation between actual and predicted stock prices. Computing this on log returns tends to become unstable or to diverge. Also trying to reconstruct prices trajectory from predicted log returns would probably lead to major deviations from ground truth, as even small errors accumulates over time. For these reasons, although MPA was an evaluation metric in [1], its informative value for this work is negligible. However, reanchoring at daily prices, as it is usually done operatively live, a more meaningful computation of MPA can be achieved. Hence, a brief analysis and visualization of this can be found in the code.

To evaluate the proposed model, comparisons against several alternatives are performed: the standalone LSTM, as well as the GRU, and the naive baseline (log return always set to zero). In addition, since the input space was expanded from 2 to 12 features, evaluation metrics are also computed on the same models with original feature set (denoted with 2F, i.e. log returns - and not prices to keep metrics comparable - and sentiment score). All models were retrained whenever the feature set changed, but without re-tuning the hyperparameters, ensuring that differences in performance primarily reflect changes in data representation rather than in model optimization. This choice is justified since the underlying log-return series remained the same across experiments. Table 2 summarizes performances of analyzed cases.

As already mentioned in a previous footnote, test period includes the late 2018 market downturn, characterized by a strong regime shift and heightened volatility. As expected, all models suffer a degradation in performance when evaluated on such conditions, having been trained on more stable regimes. Nevertheless, the proposed ensemble consistently outperforms single recurrent models in terms of error metrics (lowest MSE and NRMSE) and achieves the highest F1-score, confirming its ability to balance precision and recall. Notably, the naive baseline remains unmatched in terms of MDA, reflecting the inherent advantage of a persistence model in strongly autocorrelated series like stock prices (or equivalently log returns). However, its precision, recall, and F1-score collapse

TABLE 2: Performance comparison across models. Best values per column are highlighted in bold.

Model	MSE	NRMSE	Prec.	Recall	F1	MDA
Naive baseline	0.000176	1.004	0.000	0.000	0.000	0.585
LSTM	0.000220	1.124	0.342	0.382	0.361	0.506
GRU	0.000261	1.224	0.424	0.412	0.418	0.517
Proposed (full)	0.000175	1.001	0.447	0.618	0.519	0.524
2F LSTM	0.000288	1.286	0.359	0.412	0.384	0.504
2F GRU	0.000385	1.486	0.346	0.529	0.419	0.477
Baseline (2F)	0.000188	1.038	0.394	0.765	0.520	0.415

to zero, making it practically useless for predictive trading strategies. The two-feature framework (taken from [1]) achieves strong recall ($\sim 76\%$), but this comes at the cost of much weaker precision and directional accuracy, indicating a bias towards over-predicting positive movements. By contrast, the proposed model maintains a more balanced profile, with recall above 60%, implying a tendency to correctly capture upward movements while still preserving reasonable precision. The obtained MDA of $\sim 52\%$ is only modestly above random guessing, highlighting the difficulty of directional forecasting in highly volatile and not as extensively modeled regimes, yet it still compares favorably against all the other approaches. Finally, the normalized RMSE values close to 1 confirm that predictive errors remain comparable to the intrinsic variability of returns; indeed, NRMSE values cannot fall significantly below 1, since σ represents the natural lower bound for error scaling, hence showing that no additional noise has been learned. Overall, while not miraculous in terms of directional accuracy, the proposed modified ensemble demonstrates measurable gains in error reduction and balanced classification metrics, making it a robust improvement over prior approaches.

Further improvements could be many.

First of all, as it is often - if not always - the case in deep learning, being able to operate with more data covering a bigger time span, hence different market's conditions, could be beneficial at improving learning, cross validation and overall generalization. Otherwise a continual learning framework could be exploited, may allowing for models more resources-friendly, but surely requiring a dynamical process. Also, for the purpose of predicting daily movements in the market, "quicker" features should be deployed. By this meaning features that can rapidly and reactively capture shifts in market conditions, such as the CBOE put/call ratio (CPCE), which often fluctuates significantly between consecutive days, carrying high informative value. Sentiment-like indicators may require giving up slower-moving features, such as the 50-day moving average or the 20-day rolling volatility, which tend to reflect more structural market trends rather than short-term reversals. Exogenous features like the CPCE were not considered in this work due to the difficulty in retrieving them. Even a better news data collection could enhance model performance, since the headlines used in this work often appear to be poorly informative regarding the daily estimation of S&P500 returns. Although they mostly refer to companies belonging to the index, their relevance to the aggregate market movement is

limited, as they often concern firm-specific events that are relatively minor, or can become misleading, compared to the index as a whole.

Other interesting approaches, besides adding more and improved base learners (being careful of overfitting), include the use of fine-tuned window size and especially more meaningful losses. While MSE is a standard choice in regression tasks, it does not account for the time-varying volatility that characterizes financial series. In this regard, an interesting line of research has been proposed by Grassi [14], who investigated the use of a volatility-aware loss function. Although the empirical results of that work, based on feed-forward neural networks, were not conclusive, the idea remains valuable. Such a loss would assign greater weight to prediction errors occurring in calmer regimes, while still considering volatility magnitude, to avoid models from always predicting large variances. Exploring such losses, in combination with modern architectures like recurrent or attention-based models, may represent a promising direction for future improvements.

One final observation: although working with data is really fascinating, preprocessing becomes an highly non-trivial task when operating at large scale. It must be carried out with extreme accuracy, since results cannot be verified on a case-by-case basis, and even minor oversights may compromise the conclusions.

REFERENCES

- [1] Y. Li and Y. Pan, "A Novel Ensemble Deep Learning Model for Stock Prediction Based on Stock Prices and News," *arXiv preprint arXiv:2007.12620*, July 2020. arXiv:2007.12620.
- [2] A. Gerig, "High-frequency trading synchronizes prices in financial markets," tech. rep., U.S. Securities and Exchange Commission, Division of Economic and Risk Analysis, Jan. 2015. Working Paper. Available: <https://www.sec.gov/files/dera-wp-hft-synchronizes.pdf>.
- [3] G. E. P. Box and G. M. Jenkins, *Time Series Analysis: Forecasting and Control*. Holden-Day, 1st ed., 1970.
- [4] T. Bollerslev, "Generalized autoregressive conditional heteroskedasticity," *Journal of Econometrics*, vol. 31, pp. 307–327, Apr. 1986.
- [5] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, Sept. 1995.
- [6] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, Oct. 2001.
- [7] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, pp. 1735–1780, Nov. 1997.
- [8] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder–decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, June 2014.
- [9] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, pp. 2278–2324, Nov. 1998.
- [10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, (Long Beach, CA, USA), pp. 5998–6008, Dec. 2017.
- [11] Z. Dong, X. Fan, and Z. Peng, "Fnspsid: A comprehensive financial news dataset in time series," *arXiv preprint arXiv:2402.06698*, Feb. 2024.
- [12] D. Araci, "Finbert: Financial sentiment analysis with pre-trained language models," *arXiv preprint arXiv:1908.10063*, Aug. 2019.
- [13] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, (Vancouver, Canada), Apr. 2017.
- [14] S. Grassi, "Financial time series forecasting with neural networks," Master's thesis, Politecnico di Milano, Milan, Italy, Dec. 2018. Available at https://www.politesi.polimi.it/retrieve/a81cb05c-fb95-616b-e053-1605fe0a889a/2018_12_Grassi_Stefano.pdf.